Anomaly Detection for MITRE ATT&CK T1059

Presented by*:*

- **Yossi Okropiridze**
- **Michael Naftalishen**

Table of contents:

1. **T1059 Introduction**
2. **Synthetic data generation**
3. **Preprocessing of the data**
4. **Agents**
5. **Metrics**
6. **Workflow**
7. **Project reports**
8. **Insight**

1. Introduction to MITRE ATT&CK Technique T1059

MITRE ATT&CK technique **T1059 – Command and Scripting Interpreter** refers to the use of command-line interfaces and scripting environments to execute commands, automate tasks, and potentially carry out malicious operations within a target system. T1059 is a commonly leveraged technique in cyber-attacks because it allows adversaries to execute arbitrary code, move laterally, and manipulate system processes without relying on traditional malware binaries, thereby increasing stealth and persistence.

Command and scripting interpreters, such as **cmd.exe**, **PowerShell**, **Python**, **WScript**, and **cscript**, provide versatile interfaces for interacting with the operating system. While these tools are essential for legitimate administrative tasks, they can also be exploited to perform actions such as executing unauthorized scripts, downloading malicious payloads, or escalating privileges.

Detection of T1059-related activities poses significant challenges due to the dual-use nature of command interpreters. Normal administrative operations often resemble suspicious activity patterns, making it difficult to distinguish benign from malicious behavior. Temporal factors, user roles, and process hierarchies contribute to this complexity, requiring advanced anomaly detection and contextual analysis.

In this project, T1059 serves as the focus for **synthetic data generation and anomaly detection research**. By modeling realistic command execution behaviors, including parent-child process relationships, user roles, and temporal patterns, the study aims to develop and evaluate methods capable of identifying deviations that may indicate suspicious or malicious command execution in enterprise environments.

2. Synthetic Data Generation for MITRE ATT&CK T1059

To systematically evaluate anomaly detection methods for MITRE ATT&CK technique **T1059 (Command and Scripting Interpreter)**, synthetic datasets were generated to simulate command execution activity across multiple enterprise environments. The dataset aims to capture both benign operational behavior and malicious or anomalous command execution patterns that could indicate potential T1059 attacks.

Enterprise Profiles:

1. **Enterprise A – Small Office Scenario**
   This enterprise simulates a small office environment with a limited number of users and predictable working hours. Daily operations are mostly routine, with few variations in command usage.
   Implications for synthetic data generation:
   - High consistency makes it easier to label anomalies accurately.
   - Rare off-schedule commands are likely true anomalies.
   - Dataset can remain small while still representing realistic attack patterns.
2. **Enterprise B – Medium Company Scenario**
   A medium-sized company with more users and occasional activities outside standard hours. Operations are mostly regular but include occasional deviations due to extended projects or flexible schedules.
   Implications for synthetic data generation:
   - Moderate variability requires careful balancing of normal vs. anomalous commands.
   - Synthetic anomalies must be subtle to avoid unrealistic separation from benign actions.
   - Off-hours legitimate commands can increase false-positive risk.
3. **Enterprise C – Large 24/7 Organization**
   A large-scale enterprise operating around the clock with numerous users and high volumes of command activity.
   Implications for synthetic data generation:
   - High variability and overlapping patterns make anomaly generation more challenging.
   - Synthetic dataset must include frequent legitimate off-hours and batch operations.
   - Anomalous T1059 activity may blend with normal high-volume operations, requiring nuanced labeling.

Key Design Considerations:

- Time Distribution: Each enterprise's synthetic data respects its operational hours. Enterprise A has concentrated activity during office hours, B shows moderate off-hour activity, and C has continuous activity across all hours.

- User Diversity: User counts vary to reflect realistic environments. Small enterprises have fewer user IDs, while large enterprises introduce many users with overlapping behaviors.
- Command Frequency: Frequency distributions of commands are scaled to match enterprise size. Rare commands are treated as anomalies in small enterprises but may be common in larger enterprises.
- Anomaly Labeling: Synthetic anomalies are injected based on deviations from baseline behavior, considering time, frequency, and user patterns. The goal is to simulate realistic T1059 attacks without generating artificial noise.

This approach ensures that the generated datasets for MITRE ATT&CK T1059 are representative of real-world enterprise environments, providing a solid foundation for evaluating anomaly detection agents across varying organizational scales.

Each record in the dataset includes detailed features relevant to T1059 analysis:

- **Timestamp**: Captures execution time with configurable business hours and weekend considerations, introducing realistic temporal patterns for benign versus anomalous events.
- **Host ID and User ID**: Representing the distributed enterprise environment.
- **User Role**: Differentiating administrative versus standard user activity.
- **Process and Parent Process Names**: Critical for tracing process hierarchies.
- **Command Text**: Generated using combinatorial command structures (Get-, Set-, New-, etc.) and target objects (Process, Service, File, Registry), including variable lengths and argument counts to simulate complexity.
- **Script Type, Command Length, Number of Arguments**: Quantitative features supporting anomaly detection.
- **Execution Result**: Success or failure outcome.
- **Label (Benign vs. Suspicious)**: Assigned probabilistically with a 5% anomaly rate to model rare malicious activity.

This synthetic dataset facilitates controlled experimentation with anomaly detection algorithms, allowing researchers to evaluate detection capabilities against structured yet realistic command execution sequences. By simulating T1059 behaviors, the data supports the development and assessment of agents capable of identifying suspicious command execution patterns across diverse enterprise environments

3. Data Preprocessing for T1059 Anomaly Detection

Effective preprocessing is critical for anomaly detection in MITRE ATT&CK technique **T1059 (Command and Scripting Interpreter)**, as raw command execution data contains heterogeneous and high-dimensional features, including timestamps, user roles, process hierarchies, and textual command content. The preprocessing pipeline implemented in this project transforms the raw dataset into a structured and normalized format suitable for agent-based anomaly detection.

The preprocessing workflow includes several key steps:

1. **Label Encoding**
   The is_anomaly label is mapped to numerical values (0 for benign, 1 for suspicious) to support supervised evaluation when available.

2. **Timestamp Feature Engineering**
   Command execution timestamps are parsed into datetime objects, and multiple **cyclic temporal features** are derived to capture periodic patterns:

   - Hour, minute, day, month, and day-of-week are encoded as sine and cosine components.
   - A binary is_weekend feature identifies weekend activity.

This cyclic encoding preserves the continuity of temporal data and allows anomaly detection algorithms to recognize irregular temporal behavior, such as off-hours or unusual weekday activity, which is particularly relevant for detecting T1059 exploitation.

3. **Handling Missing Values**
   Missing numeric values are replaced with median values, while categorical features are filled with 'Unknown'. Command text is sanitized to avoid null entries. This ensures consistent data quality for all agents.

4. **Correlation Reduction**
   Highly correlated numeric features (above a configurable threshold, default 0.95) are removed to reduce redundancy and prevent bias in distance- or variance-based detection methods.

5. **Text Cleaning and TF-IDF Vectorization**
   The command_text column is cleaned by lowercasing, removing non-ASCII characters, and normalizing whitespace. TF-IDF vectorization is applied to represent command content as high-dimensional numeric features, capturing the lexical patterns of T1059 command sequences, including potentially suspicious scripts or unusual command arguments.

6. **Categorical Feature Encoding**
   Categorical features such as host IDs, user IDs, and roles are frequency-encoded to convert them into numeric values while preserving the relative importance of frequent versus rare categories.

7. **Feature Scaling and Combination**
   Numeric features are standardized using StandardScaler, and TF-IDF vectors are concatenated to produce a unified feature matrix. This ensures all features contribute proportionally during anomaly detection.

8. **Preparation for Multi-Agent Analysis**
   The final feature matrices are packaged into a dictionary for multiple agents (Isolation Forest, Autoencoder, One-Class SVM), allowing consistent evaluation across different anomaly detection algorithms.

This preprocessing pipeline transforms heterogeneous enterprise command execution data into a **robust, normalized, and high-dimensional representation** suitable for detecting suspicious command and scripting behavior associated with T1059. By combining temporal, numeric, categorical, and textual features, the system can capture complex patterns of both normal and anomalous activity, providing a foundation for accurate agent-based anomaly detection.

4. Anomaly Detection Agents Overview:

4.1. BaseAgent – Abstract Agent Framework

The BaseAgent serves as the foundational abstract class for all anomaly detection agents in this project. It establishes a common interface and standardized workflow, ensuring consistency across different detection algorithms while allowing flexibility in implementing algorithm-specific logic. Key components of the BaseAgent include:

1. **Initialization (__init__)**
   Each agent is assigned a name attribute for identification, and a model attribute is reserved for the underlying detection algorithm, which will be implemented in derived classes.
2. **Abstract Methods**
   - fit(X): This method is designed to train the agent's model using feature matrix X. Each specific agent (e.g., Isolation Forest, Autoencoder, One-Class SVM) implements this method according to its algorithm.
   - score(X): This method computes anomaly scores for the input features. Scores are numerical values where a higher score indicates a more anomalous observation. The BaseAgent does not define the computation itself; it delegates to concrete subclasses.
3. **Prediction (predict)**
   The predict method allows conversion of continuous anomaly scores into binary labels (benign vs. anomalous) based on an optional threshold. If no threshold is provided, the raw anomaly scores are returned. This standardizes the output across agents, enabling comparison and ensemble aggregation.
4. **Utility (get_name)**
   Returns the agent's name, facilitating identification in multi-agent analyses and ensemble evaluations.
   By enforcing a uniform interface, the BaseAgent ensures that all derived agents can be interchangeably used in the anomaly detection pipeline. This design supports structured experimentation with multiple detection methods while maintaining compatibility with the synthetic T1059 datasets, allowing consistent evaluation of suspicious command and scripting behavior across diverse agent models.

4.2. AutoencoderAgent – Neural Network-Based Detection

The **AutoencoderAgent** is a neural network-based anomaly detector designed to identify irregular command and scripting behavior, consistent with MITRE ATT&CK T1059 activities. Unlike classical statistical methods, the autoencoder leverages **unsupervised representation learning** to capture the underlying structure of benign command sequences and detect deviations indicative of suspicious actions.

Key Components and Methodology:

1. **Architecture**

   The model is structured as a fully connected autoencoder consisting of:
   - An input layer matching the dimensionality of preprocessed features.
   - Hidden layers with progressively smaller dimensions (configurable, e.g., 64 → 32).
   - A latent representation layer (e.g., 16 neurons) that compresses the feature space, effectively encoding the normal behavior patterns.
   - A decoder that reconstructs the original input from the latent representation, mirroring the encoder structure.

   The network uses **ReLU activations** in hidden and latent layers, with a linear output layer to predict the original input features.
2. **Training Procedure**
   - The model is trained **unsupervised** using the **mean squared error (MSE)** between the original input and its reconstruction as the loss function.
   - Input features are **standardized** using a StandardScaler to ensure stable training.
   - The agent supports **early stopping** based on validation loss, preventing overfitting and ensuring the model generalizes well to unseen benign behavior.
   - Reproducibility is enforced through fixed random seeds across Python, NumPy, and TensorFlow.
3. **Anomaly Scoring**
   - After training, the agent computes **reconstruction errors** for new inputs.
   - Observations with high reconstruction error are flagged as anomalous, as they deviate significantly from the patterns learned from benign commands.
   - The score reflects the degree of anomaly: **higher MSE indicates greater deviation**.
4. **Thresholding and Prediction**
   - Binary classification (benign vs. suspicious) requires a **threshold**, typically determined externally (e.g., via MetaAgent or cross-validation).
   - This allows the AutoencoderAgent to flexibly integrate with ensemble methods or standalone anomaly analysis.

Relevance to T1059

By modeling normal command execution sequences (e.g., powershell.exe, cmd.exe, python.exe) and their arguments, the AutoencoderAgent can detect **unusual command patterns, unexpected parent-child process relationships, or irregular execution times**, which are indicative of **malicious scripting activity** under T1059. This capability makes it a powerful tool for uncovering subtle, previously unseen anomalies in enterprise environments.

4.3. IsolationForestAgent – Tree-Based Anomaly Detection

The IsolationForestAgent implements a tree ensemble method specifically designed for anomaly detection. Unlike distance- or density-based approaches, Isolation Forest isolates anomalies rather than profiling normal data, making it effective for detecting rare and irregular command behaviors typical of MITRE ATT&CK T1059 activities.

Key Components and Methodology:

1. **Architecture**
    - The agent uses the Isolation Forest algorithm, which constructs an ensemble of random decision trees to partition the feature space.
    - Anomalies are easier to isolate because they differ significantly from most data points, often requiring fewer splits to separate from the rest of the dataset.
    - This property allows the model to efficiently identify outliers in high-dimensional numeric spaces, such as command feature vectors including timestamps, numeric encodings, and TF-IDF representations of command text.
2. **Training Procedure**
    - The model is trained on numeric features derived from preprocessing.
    - Configurable hyperparameters include:
        - n_estimators: the number of trees in the forest.
        - max_samples: the number of samples used to build each tree.
        - contamination: an estimate of the proportion of anomalies in the dataset.
    - Each tree is constructed by randomly selecting a feature and a split value, recursively partitioning the data until all points are isolated.
3. **Anomaly Scoring**
    - The agent computes an anomaly score for each observation based on the path length required to isolate it in the trees.
    - Shorter path lengths correspond to more anomalous behavior.
    - Scores are flipped so that higher values indicate higher anomaly likelihood, aligning with standard interpretation in ensemble evaluation.
4. **Thresholding and Prediction**
    - If no explicit threshold is provided, the agent uses the contamination parameter to automatically determine a cutoff for outlier detection.
    - This makes the agent robust in scenarios where anomaly frequency is approximately known or when integrating into a broader ensemble framework.

Relevance to T1059

The IsolationForestAgent is particularly suited for detecting rare or unusual scripting activity, for instance, unexpected PowerShell or cmd.exe commands, anomalous argument patterns, or abnormal execution timing—without requiring explicit labeling. Its ability to identify subtle deviations in complex, high-dimensional feature spaces make it an effective tool for uncovering malicious command and scripting behavior, as described under MITRE ATT&CK T1059.

4.4. SVMAgent – One-Class SVM Anomaly Detection

The SVMAgent implements a One-Class Support Vector Machine (SVM) for anomaly detection. This approach is particularly effective for identifying outliers in high-dimensional numeric feature spaces, such as those generated from preprocessed command logs and TF-IDF representations of command text.

Key Components and Methodology:

1. **Architecture**
    - The agent employs the One-Class SVM, a kernel-based method that learns the boundary of normal data in the feature space.
    - By mapping input features into a higher-dimensional space (via the chosen kernel, e.g., rbf), the model can capture complex distributions of normal command behavior.
    - Observations falling outside this learned boundary are considered anomalies, making it suitable for detecting rare or unusual command activity associated with MITRE ATT&CK T1059.
2. **Training Procedure**
    - The model is trained exclusively on numeric features, which may include timestamp encodings, frequency-encoded categorical features, and TF-IDF vectors.
    - Key hyperparameters include:
        - nu: an upper bound on the fraction of training errors and a lower bound on the fraction of support vectors.
        - kernel: specifies the kernel type (default is RBF).
        - gamma: kernel coefficient that controls the influence of single training samples.
    - The SVM fits a tight boundary around the normal data, effectively isolating anomalies in the feature space.
3. **Anomaly Scoring**
    - The agent computes anomaly scores via the SVM's decision function, where positive values indicate inliers and negative values indicate outliers.
    - Scores are flipped so that higher values correspond to higher anomaly likelihood, making interpretation consistent across different agents in the ensemble.
4. **Thresholding and Prediction**
    - If no explicit threshold is provided, the agent determines a cutoff using the contamination parameter to approximate the expected fraction of anomalies.
    - This allows the agent to output binary predictions compatible with ensemble aggregation or MetaAgent decision-making.

Relevance to T1059

The SVMAgent is well-suited for detecting abnormal command execution patterns such as suspicious scripting, unusual arguments, or unexpected timing. Its kernel-based boundary learning allows it to identify subtle deviations in high-dimensional numeric spaces, complementing other agents like Isolation Forest and Autoencoder. This makes it a valuable component in a multi-agent anomaly detection system targeting MITRE ATT&CK T1059 techniques.

4.5. MetaAgent – Multi-Agent Anomaly Aggregation

The MetaAgent serves as a meta-level ensemble that aggregates multiple anomaly detection agents, combining their strengths to improve overall detection performance. It is designed to integrate agents like Isolation Forest, One-Class SVM, and Autoencoder, handling both raw numeric and TF-IDF features via the X_dict input.

Key Components and Methodology:

1. **Architecture**
    - The MetaAgent accepts a list of pre-initialized agents that inherit from the BaseAgent interface.
    - Each agent operates independently on its designated feature subset, allowing for heterogeneous data processing (e.g., numeric-only for SVM and Isolation Forest, scaled + TF-IDF for Autoencoder).
2. **Voting Mechanisms**
    - Soft Voting (default):
        - Computes a weighted sum of the anomaly scores from all agents.
        - Weights can be uniform or explicitly provided, allowing prioritization of more reliable agents.
        - Produces a continuous final score, where higher values indicate higher anomaly likelihood.
    - Hard Voting:
        - Converts each agent's scores into binary predictions using percentile-based thresholds derived from the contamination parameter.
        - Final prediction is the majority vote, expressed as the fraction of agents that classify a sample as anomalous.
3. **Training Procedure**
    - The MetaAgent delegates training to each agent individually.
    - For agents requiring validation data (e.g., Autoencoder), an optional X_val_dict can be provided.
    - This design ensures agents like Autoencoder can employ early stopping based on validation loss, while others train directly on their numeric feature sets.
4. **Score Computation and Prediction**
    - score(X_dict) returns the final anomaly scores, integrating all agents according to the chosen voting strategy.
    - predict(X_dict, threshold=None) outputs binary anomaly labels:

    - If no threshold is specified, the MetaAgent automatically calculates a threshold using the contamination rate for soft voting or majority vote for hard voting.
    - Explicit thresholds can also be applied for custom sensitivity.

Advantages and Relevance to T1059

The MetaAgent provides a robust ensemble detection system for MITRE ATT&CK T1059, capable of identifying both obvious and subtle anomalous command executions. By combining the diverse detection strategies of its agents, it can:

- Capture high-dimensional deviations in numeric and text features.
- Reduce false positives by relying on agreement across agents.
- Adapt to different anomaly patterns, from unusual scripting to suspicious system commands.

This makes the MetaAgent a central orchestrator in a multi-agent anomaly detection pipeline, ensuring comprehensive coverage of malicious command activity.

5. Evaluation and Metrics

The evaluation_utils.py module provides utility functions to evaluate both individual agents and ensembles of anomaly detection agents. The goal is to assess how well each agent, or the ensemble can detect anomalies in the dataset.

1. Evaluating a Single Agent (evaluate_agent)

This function takes an agent instance, feature data X, true labels y_true, and an optional threshold.

Steps:

1. Prediction:
    - If a threshold is provided, the agent will use it to make binary anomaly predictions.
    - If no threshold is provided, the agent's internal logic (e.g., automatic thresholding in MetaAgent or contamination-based threshold) is used.
2. Metrics Computation:
   Using y_true and y_pred, the function calculates:
    - Confusion Matrix (CM): Shows the counts of True Positives, True Negatives, False Positives, and False Negatives.
    - Precision: The proportion of predicted anomalies that were actually anomalies.

$$\text{Precision} = \frac{TP}{TP + FP}$$

    - Recall: The proportion of actual anomalies that were correctly detected.

$$\text{Recall} = \frac{TP}{TP + FN}$$

    - F1-Score: Harmonic mean of precision and recall, balancing both.

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

3. Output:
   Prints the confusion matrix and the computed metrics, and returns them as a dictionary for further processing.

---

2. Evaluating an Ensemble of Agents (evaluate_ensemble)

This function evaluates a collection of agents working together (like the MetaAgent approach).

Steps:

1. Score Aggregation:

    - Each agent computes its anomaly scores on its respective feature set.
    - Scores from all agents are combined using a weighted sum. If weights are not provided, all agents are equally weighted.
2. Thresholding:
    - If no explicit threshold is given, a percentile-based threshold is used based on the expected contamination rate (e.g., top 5% scores are considered anomalies).
3. Metrics Computation:
   Same as for a single agent: confusion matrix, precision, recall, F1-score.
4. Output:
   Prints ensemble evaluation results and returns a dictionary containing:
    - Confusion matrix
    - Precision, recall, F1
    - Threshold used
    - Predicted labels (0 for normal, 1 for anomaly)

---

3. Key Points About Metrics in Anomaly Detection

- Confusion Matrix: Provides a complete view of detection performance, including both correct and incorrect classifications.
- Precision vs Recall: There is a trade-off:
    - High precision → few false positives (normal points incorrectly flagged as anomalies).
    - High recall → few false negatives (anomalies missed).
- F1-Score: Useful when the dataset is imbalanced (anomalies are rare), which is typical in cybersecurity datasets like command execution anomalies (T1059).

6. Workflow of the Anomaly Detection System

Workflow of the Anomaly Detection System

The main.py script orchestrates the entire anomaly detection workflow, from dataset selection to final reporting. The process integrates preprocessing, agent training, evaluation, and ensemble aggregation. Below is a detailed step-by-step overview.

---

1. Dataset Selection

- The system allows the user to select one of the enterprise datasets (A, B, C) via the best_hyperparams.select_dataset() function.
- The results folder for storing outputs is automatically created.

---

2. Data Loading

- The selected dataset is loaded using pandas.read_csv.
- The dataset typically includes:
    - command_text (textual commands executed)
    - timestamp (when each command was executed)
    - is_anomaly (binary label: 0 = benign, 1 = suspicious)

---

3. Preprocessing

- Data is processed through preprocess_for_metaagent():
    1. **Text processing**: TF-IDF vectorization of command_text.
    2. **Timestamp cyclic encoding**: hour, minute, day-of-week, month features transformed using sine/cosine.
    3. **Numeric scaling**: all numeric features standardized with StandardScaler.
    4. **Missing values handling**: median imputation for numeric features, Unknown for categorical.
    5. **Highly correlated feature removal**: drop features with correlation above 0.95.
- Output: X_dict (feature arrays for each agent) and y (labels).

---

4. Train / Validation / Test Split

- Each agent receives its own split of data:
    - **Train**: for model fitting

    - **Validation**: for threshold tuning and hyperparameter selection
    - **Test**: for final evaluation
- Special handling for Autoencoder:
    - Training is performed only on **benign samples** to model normal behavior.

---

5. Agent Initialization

- Individual agents are created:
    - **IsolationForestAgent**: parameters tuned using find_best_n_estimators_if().
    - **SVMAgent (One-Class SVM)**: nu tuned using find_best_nu().
    - **AutoencoderAgent**: input dimension derived from features, latent dimension and epochs configured.
- All agents are collected in a list for ensemble evaluation.

---

6. MetaAgent Initialization

- A MetaAgent aggregates the three individual agents.
- Supports **soft voting** (weighted sum of anomaly scores) or **hard voting** (majority vote based on thresholds).
- Initial weights are set (default equal or customized).

---

7. Training / Fitting

- Each agent is trained on its corresponding training data.
- Validation sets are provided for agents that require it (e.g., Autoencoder uses validation loss for early stopping).

---

8. Individual Agent Evaluation

- Each agent is evaluated on the **validation set**:
    - Find the **best threshold** that maximizes F1-score.
    - Evaluate **Precision, Recall, F1**, and **Confusion Matrix**.
    - Generate graphs for visualization.
- The metrics and graphs are stored for reporting.

---

9. MetaAgent Ensemble Evaluation

- The system finds the **best ensemble weights** and **threshold** on the validation set.
- Evaluation of **test set** includes:
  - Weighted aggregation of scores from all agents
  - Threshold-based anomaly prediction
  - Computation of **ensemble metrics** (Precision, Recall, F1)
  - Generation of confusion matrix graph
- Ensures robust combination of agents for final predictions.

---

10. Result Saving

- Final anomaly scores and predictions are saved to a CSV file.
- A **PDF report** is generated, containing:
  - Individual agent performance
  - Ensemble metrics
  - Confusion matrix visualizations
- Temporary graph PNG files are deleted after report generation.
- Metrics are also exported as a CSV summary.

---

11. Enterprise Comparison

- If multiple enterprise datasets are analyzed, the workflow can compare performance across datasets using the compare_enterprises() utility.

Key Notes

- The workflow ensures **reproducibility** (fixed random seeds, deterministic Autoencoder).
- Hyperparameter tuning is integrated with validation-based optimization.
- Both **single-agent evaluation** and **ensemble evaluation** are included for a complete analysis.
- Supports **scalable and modular architecture**: new agents can be added easily.

7. Reporting System

The reporting utilities in report_generator.py provide a complete mechanism to document and visualize the performance of anomaly detection agents and their ensemble. The system supports:

- Exporting metrics to CSV
- Generating PDF reports with graphs and agent metrics
- Visualizing confusion matrices
- Comparing results across multiple enterprise datasets

---

1. Metrics CSV Export

- Function: export_metrics_to_csv(filename, agents_info, ensemble_info=None)
- Purpose:
  - Saves **Precision, Recall, and F1-score** for each agent and optionally the ensemble.
  - Creates a structured CSV that can be used for external analysis or reporting.
- Inputs:
  - agents_info: list of dictionaries with name and metrics
  - ensemble_info: dictionary with ensemble metrics
- Output: CSV file containing all agent and ensemble metrics.
- Example CSV columns:
  - Agent name
  - Recall
  - Precision
  - F1 Score

---

2. PDF Report Generation

- Function: generate_pdf_report(filename, agents_info, ensemble_info, weights=None)
- Purpose:
  - Generates a **professional PDF report** summarizing:
    - Each agent's metrics (Precision, Recall, F1)
    - Confusion matrix graphs
    - Ensemble metrics with **weights** assigned to each agent
  - Automatically handles layout and page breaks for large reports

- Features:

1. **Agent Sections**: Each agent gets a dedicated section with metrics and graph.
2. **Ensemble Section**: Includes metrics, graph, and display of weights for soft voting.
3. **Graph Handling**: Confusion matrix graphs are resized to fit page constraints.

- Output: PDF file with clear visualization and metrics summary.

---

3. Confusion Matrix Visualization

- Function: plot_confusion_matrix_graph(y_true, y_pred, title, path)
- Purpose:
  - Generates a **visual representation of the confusion matrix**.
  - Colors and text display make it easy to read true positives, false positives, false negatives, and true negatives.
- Output: PNG image file saved to disk.
- Used in both individual agent reports and ensemble reports.

---

4. Enterprise Comparison

- Function: compare_enterprises(results_folder="Results")
- Purpose:
  - Compares performance metrics across multiple enterprise datasets (A, B, C).
  - Visualizes **Recall, Precision, F1-score** for each agent across datasets.
- Workflow:

1. Loads CSV metrics from each enterprise.
2. Generates bar plots for each metric with agents on x-axis and enterprises as grouped bars.
3. Saves graphs in Results/Enterprise_Comparison.

- Outcome:
  - Provides **visual insights into which agents perform consistently** across different datasets.
  - Useful for cross-enterprise benchmarking and reporting.

Key Features of the Reporting System

1. **Automatic Metrics Capture**: Precision, Recall, F1 are automatically calculated for agents and ensembles.

2. **Graphical Visualization**: Confusion matrices and comparison plots improve interpretability.
3. **PDF Integration**: Combines metrics and visualizations in a single document.
4. **Flexible Weight Display**: Ensemble section shows weights used for soft voting.
5. **Cross-Enterprise Analysis**: Allows for straightforward comparison across multiple datasets, supporting enterprise-level performance evaluation.

This reporting system ensures that all experiment results are **reproducible, visually clear, and ready for presentation**—ideal for research reports or technical documentation.

8. Key Insights from Anomaly Detection Results

1. Enterprise A

- **IsolationForest**: Balanced performance with moderate precision (0.6250) and recall (0.7143). Good general detection of anomalies but prone to some false positives.
- **OneClassSVM**: Very high recall (1.0000), excellent at catching all anomalies, but precision (0.7000) indicates some false positives.
- **Autoencoder**: Low precision (0.4545) and moderate recall (0.7143), suggesting it flagged many normal samples as anomalies.
- **Ensemble**: Weighted combination favored IsolationForest (0.80) and OneClassSVM (0.20), Autoencoder ignored (0.00). Resulting ensemble achieved perfect recall (1.0000) and F1 of 0.7000, balancing detection coverage with fewer false positives than Autoencoder alone.

Insight: IsolationForest drives most decisions, while OneClassSVM ensures no anomalies are missed. Autoencoder's contribution was minimal due to lower precision.

2. Enterprise B

- **IsolationForest**: Strong precision (0.8421) and solid recall (0.7273), performing well on clear anomaly patterns.
- **OneClassSVM**: Perfect precision (1.0000) but lower recall (0.6364), detecting fewer anomalies but very accurately.
- **Autoencoder**: Moderate recall (0.7727) but very low precision (0.3864), resulting in many false positives.
- **Ensemble**: Weighted heavily toward OneClassSVM (0.80) and some Autoencoder contribution (0.20). Ensemble achieved perfect precision (1.0000) and F1 of 0.8108, slightly improving over individual agents.

Insight: For Enterprise B, accuracy in labeling true anomalies was crucial, so the ensemble favored precision via OneClassSVM, with Autoencoder providing minor additional coverage. IsolationForest was ignored (0.00 weight), showing its redundancy for this dataset.

3. Enterprise C

- **IsolationForest**: Modest precision (0.5909) and recall (0.6265), not very strong alone.
- **OneClassSVM**: Higher precision (0.8136) but lower recall (0.5783), detecting fewer anomalies but more accurately.
- **Autoencoder**: High precision (0.7500) but very low recall (0.3976), conservative but misses many anomalies.
- **Ensemble**: Balanced weights between OneClassSVM (0.40) and Autoencoder (0.60) produced F1 0.6906, precision 0.8571, and recall 0.5783. The ensemble outperformed any individual agent in precision while maintaining acceptable recall.

Insight: Enterprise C required combining Autoencoder and SVM for complementary detection: Autoencoder provided stable detection for some anomalies, while OneClassSVM captured others missed by Autoencoder. IsolationForest contributed little and was ignored (0.00 weight).

4. General Observations Across Enterprises

1. **MetaAgent is consistently superior**:
    - Improves F1-score in all enterprises compared to most individual agents.
    - Adaptive weighting selects the most relevant agents for each dataset.
2. **Agent contributions are dataset-dependent**:
    - IsolationForest dominates in Enterprise A.
    - OneClassSVM dominates in Enterprise B.
    - Combination of OneClassSVM and Autoencoder is optimal in Enterprise C.
3. **Trade-offs between precision and recall**:
    - High recall (catch all anomalies) often lowers precision (more false positives).
    - Ensemble weights effectively manage this trade-off per dataset.
4. **Autoencoder's role varies**:
    - Can help in high-dimensional, subtle anomalies (Enterprise C).
    - Can be ignored when precision is crucial (Enterprise A and B).

Key Takeaways

- **Weighting is critical**: The MetaAgent dynamically adjusts agent influence to maximize F1, adapting to dataset characteristics.
- **No single agent is universally best**: Each dataset benefits from a different agent combination.
- **Ensemble ensures robust detection**: By aggregating complementary strengths, the ensemble avoids relying on any single agent's weaknesses.